

**Gestión y Clasificación de Líneas de Producción Industriales Mediante Operaciones de
Conjuntos y Lógica Proposicional en C++.**

Julio Andrade Chicaiza, Edwin Guachi Cabrera, Mikael Hidalgo Nogales, Erick Sabando
Castillo, Anthony Vilaña Quishpe.

Departamento de Ciencias de la Computación, Universidad de las Fuerzas Armadas ESPE,
Sangolquí, Ecuador.

Resumen

Este documento presenta el diseño e implementación de un sistema computarizado orientado a la gestión y categorización de líneas de producción dentro de contextos industriales. Dicho sistema facilita el registro individualizado de cada línea productiva, incorporando métricas relevantes como el volumen de producción logrado, los objetivos de rendimiento diario, la incidencia de fallas mecánicas y la aprobación del supervisor. Con base en esta información, el software efectúa una evaluación automatizada del estado operativo de cada línea, permitiendo su clasificación en categorías como 'operativa', 'con rendimiento subóptimo' o 'inactiva'. Este proceso de categorización se fundamenta en un conjunto de reglas lógicas elementales que modelan el comportamiento previsible en un entorno de planta industrial. Para estructurar la información, el sistema organiza las líneas en tres grupos distintos según su estado operativo. Posteriormente, habilita la ejecución de operaciones entre estos conglomerados, tales como la fusión de líneas categorizadas como de rendimiento subóptimo e inactivas para el análisis de aquellas con incidencias, o la identificación de líneas que satisfacen múltiples criterios simultáneamente. Adicionalmente, el software registra un historial de las acciones efectuadas y gestiona una cola de espera para las líneas que demandan una revisión. La validación del sistema se llevó a cabo mediante diversos escenarios simulados, lo que permitió constatar la correcta categorización de cada línea y la ejecución sin incidencias de las operaciones intergrupales. Se infiere que la implementación de estructuras específicas para la organización de datos, aunada a la aplicación de reglas lógicas definidas, representa una herramienta de valor y eficacia para el soporte a la toma de decisiones en el ámbito del control de la producción industrial.

Palabras clave: gestión de la producción, categorización de estados, agrupación de información, automatización, soporte a la decisión.

LÓGICA PROPOSICIONAL EN LÍNEAS DE PRODUCCIÓN

El proyecto se enfocará en el desarrollo de un sistema de control de producción industrial aplicado a la empresa **TechFactory**. Mediante implementaciones de matemáticas discretas tales como lógica proposicional, teoría de conjuntos y la lógica de predicados las cuales nos ayudaran a automatizar la toma de decisiones operativas.

El sistema va a utilizar estructura de datos dinámicos en C++ como listas, pilas y colas para lograr gestionar la información de las líneas de producción permitiendo así clasificar su estado y evaluar de mejor manera el desempeño bajo un razonamiento lógico.

Objetivos

Objetivo General

Desarrollar un sistema de control de producción con la herramienta C++ la cual aplique fundamentos de lógica proposicional, conjuntos y estructuras de datos dinámicas para la supervisión y el análisis de la eficiencia operativa de las líneas de ensamblaje la cual deberá permitir una mejor gestión en TechFactory (Planta Industrial)

Objetivos Secundarios

- Modelar y evaluar proposiciones lógicas para representar con precisión las operaciones de cada línea (Metas de producción, fallos mecánicos, etc.)
- Aplicar reglas de inferencia lógica para poder deducir si una línea se encuentra operando o si está en revisión basándose en condiciones ingresadas
- Implementar operaciones de conjuntos para clasificar las líneas en estados de rendimiento como por ejemplo alto, bajo o detenido

Marco Teórico**Estructuras de Datos lineales**

Las estructuras de datos lineales organizan los elementos de manera secuencial, donde cada componente tiene un predecesor y un sucesor único (a excepción del primer y último elemento). Esta linealidad permite un acceso y manipulación estructurada de la información en memoria, lo cual es fundamental para el diseño y análisis de algoritmos computacionales (Johnsonbaugh, 2005).

Listas Enlazadas: A diferencia de los arreglos estáticos que requieren bloques de memoria contiguos, las listas enlazadas son estructuras dinámicas compuestas por nodos. Cada nodo contiene un valor y un puntero que indica la dirección de memoria del siguiente nodo. Este diseño permite la inserción y eliminación de elementos en tiempo de ejecución sin reasignar toda la estructura.

Pilas (LIFO): Una pila es una colección donde las operaciones de inserción y extracción se realizan por un único extremo. El último elemento en ingresar es el primero en salir. Este comportamiento es esencial en la evaluación de expresiones matemáticas y en el control de llamadas a funciones (Johnsonbaugh, 2005).

Colas (FIFO): Operan bajo el principio opuesto a las pilas: el primer elemento en entrar es el primero en salir. Las inserciones se realizan al final de la estructura, mientras que las extracciones ocurren en el frente. Es la base teórica para la gestión de recursos compartidos en sistemas operativos.

Lógica Proposicional

La lógica proposicional proporciona el fundamento matemático y formal para evaluar la validez de los argumentos mediante el análisis de sentencias declarativas, siendo la base para el control de flujo en la programación de software.

Proposiciones Atómicas: Son enunciados declarativos que pueden ser evaluados bajo un valor de verdad estricto: verdaderos o falsos, pero no ambos simultáneamente (Johnsonbaugh, 2005, p. 2). Convencionalmente, se representan mediante letras mayúsculas como P, Q y R.

Conectores Lógicos: Para construir proposiciones compuestas a partir de variables atómicas, se emplean operadores lógicos que alteran o combinan sus valores de verdad. Los conectores fundamentales incluyen la conjunción ($\wedge$), la disyunción ($\vee$), la negación ($\neg$), la implicación ($\rightarrow$) y el bicondicional ($\leftrightarrow$) (Johnsonbaugh, 2005, pp. 3-6).

Tablas de Verdad Implícitas: Son herramientas analíticas exhaustivas que determinan el valor de verdad resultante de una proposición compuesta evaluando todas las combinaciones posibles de los valores de verdad de las variables atómicas involucradas (Johnsonbaugh, 2005, p. 4).

Teoría de Conjuntos

La teoría de conjuntos estudia las propiedades y relaciones de colecciones bien definidas de objetos, denominados elementos. En el contexto computacional, es el sustento para definir estructuras de bases de datos relacionales.

Unión ($A \cup B$): Es la operación matemática que resulta en un nuevo conjunto conformado por todos los elementos que pertenecen al conjunto A, al conjunto B,

o a ambos (Johnsonbaugh, 2005, p. 57). Representa la agrupación total sin permitir duplicidad.

Intersección ($A \cap B$): Genera un conjunto formado de manera exclusiva por los elementos que tienen en común el conjunto A y el conjunto B (Johnsonbaugh, 2005, p. 57). Equivale a evaluar condiciones donde múltiples criterios deben cumplirse al mismo tiempo.

Diferencia ($A - B$): Produce un conjunto que contiene todos los elementos que pertenecen estrictamente al conjunto A, excluyendo cualquier elemento que también esté presente en el conjunto B (Johnsonbaugh, 2005, p. 58).

Diseño y Arquitectura del Sistema

Modelado de Datos

El sistema está estructurado por cinco módulos principales que implementan la estructura de lista enlazada, a continuación, se describe cada clase o estructura y como aporta al programa.

Nodo “LineaProduccion”.

Este nodo representa una línea de producción, el elemento básico del programa, el cual almacena la información de variables para datos, variables lógicas las cuales se utilizarán para aplicar la lógica proposicional y un puntero para referenciar al nodo siguiente, los cuales se describen en la siguiente tabla:

Tabla #

Variables del nodo línea de producción.

Tipo de variable	Variable	Tipo de dato	Descripción breve
Variable para datos	<i>nombre</i>	string	Identifica la línea.
	<i>produccionActual</i>	double	Muestra las unidades producidas en el día.
	<i>metaDiaria</i>	double	Almacena la meta de producción.
	<i>fallosMecanicos</i>	int	Número de fallos registrados.
Variables para lógica proposicional	<i>supervisorValido</i>	bool	Indica si el supervisor validó la línea.
	<i>P, Q, R</i>	bool	Proposiciones atómicas para inferencia.
	<i>estadoInferido</i>	string	Almacena el resultado de la inferencia .
Variables para referencia	<i>siguiente</i>	LineaProduccion*	Puntero para referenciar al nodo siguiente.

Clase “ListaLineas”.

Gestiona un conjunto de líneas de producción, es decir, una lista de líneas. Esta implementa atributos y métodos propios para manejar la lista de líneas, estos son:

Tabla #.

Atributos y métodos de la clase “ListaLineas”.

Tipo de variable	Variable	Tipo de Dato o Retorno	Descripción breve
Atributos	<i>cabeza</i>	LineaProduccion	Guarda la dirección del primer nodo.
	<i>cantidad</i>	int	Cuenta la cantidad de elementos de la lista.
Métodos Principales	<i>insertar (LineaProduccion *nueva)</i>	void	Agregar una nueva línea al final de la lista
	<i>buscar (const string &nombre)</i>	LineaProduccion	Busca una línea dado su nombre

Clase Pila.

Esta clase implementa una estructura de tipo pila, en la que se irán almacenando los mensajes que registran el historial de las operaciones.

Tabla #.

Atributos y métodos de la clase “Pila”.

Tipo de variable	Variable	Tipo de Dato o Retorno	Descripción breve
Atributos	<i>tope</i>	NodoPila*	Guarda la dirección del primer nodo de toda la pila.
	<i>tamano</i>	int	Almacena el número actual de elementos en la pila.

Métodos Principales	<i>push(const string &msg)</i>	void	Método para ingresar un nodo al principio.
	<i>pop()</i>	string	Método para eliminar el primer elemento.
	<i>estaVacia()</i>	bool	Valida si la pila está vacía.
	<i>getTamanio()</i>	int	Obtiene la cantidad de nodos que representan mensajes almacenados.
	<i>verTope()</i>	string	Obtiene el primer elemento de la lista.

Clase Cola.

Esta clase implementa una estructura de tipo cola, en la que se gestiona el orden de atención a líneas que no están funcionando.

Tabla #.

Atributos y métodos de la clase “Cola”.

Tipo de variable	Variable	Tipo de Dato o Retorno	Descripción breve
Atributos	<i>frente</i>	NodoCola*	Guarda la dirección del primer nodo de la cola.
	<i>final_</i>	NodoCola*	Guarda la dirección del último nodo de la cola.
	<i>tamanio</i>	int	Representa el número actual de elementos en la cola.
Métodos Principales	<i>encolar(const string &nombre)</i>	void	Método para ingresar elemento al final.
	<i>desencolar()</i>	string	Metodo para eliminar elemento de del principio
	<i>estaVacia()</i>	bool	Valida si la cola está vacía.
	<i>getTamanio()</i>	int	Obtiene la cantidad de nodos que representan las líneas pendientes.

Clase Conjuntos.

Esta clase va a almacenar conjuntos de líneas previamente clasificadas.

Tabla #.

Atributos y métodos de la clase “Conjuntos”.

Tipo de variable	Variable	Tipo de Dato o Retorno	Descripción breve
Atributos	<i>elementos</i>	LineaProduccion**	Arreglo dinámico de punteros.
	<i>tamano</i>	int	Almacena el número actual de elementos del conjunto.
	<i>capacidad</i>	int	Guarda el numero de nodos máximo del arreglo
	<i>nombre</i>	string	Guarda el nombre del conjunto.
Métodos Principales	<i>agregar(LineaProduccion *linea)</i>	void	Método para agregar línea al conjunto si hay capacidad.
	<i>limpiar()</i>	void	Método para vaciar el conjunto.
	<i>contiene(const string &nombre)</i>	bool	Método para buscar si una línea ya pertenece al conjunto
	<i>imprimir()</i>	void	Método para mostrar elementos del conjunto

Lógica de Inferencia y Conjuntos

En este informe, la lógica de inferencia nos permite conocer el posible estado de cada línea de producción a través de las proposiciones simples P, Q ,R. Su significado gramatical y en lenguaje C++se presenta a continuación.

Tabla #.

Significado gramatical y en código de las preposiciones simples P, Q, R.

Proposición	Significado	Sentencias
P	La línea cumple con la meta de producción diaria	<i>produccionActual >= metaDiaria</i>
Q	No se registran fallos mecánicos.	<i>fallosMecanicos == 0</i>

R	El supervisor ha validado la operación.	<i>supervisorValido == true</i>
----------	---	---------------------------------

Dado el significado de las proposiciones simples, pasamos a las reglas de inferencia que el programa aplica para clasificar las líneas:

Tabla #.

Reglas de inferencia para conocer el estado de las líneas de producción P, Q, R.

Regla de inferencia	Significado
$P \wedge Q \wedge R$	La línea está operativa.
$\sim P \wedge Q$	La línea tiene bajo rendimiento.
$\sim Q$	La línea se encuentra detenida.

Además, para gestionar con juntos de líneas de producción mediante operaciones lógicas utilizaremos los conjuntos presentados a continuación.

Tabla #.

Notación de los conjuntos utilizados en el programa y su significado.

Nombre del Conjunto	Significado
A	Líneas activas.
B	Líneas con bajo rendimiento.
C	Líneas detenidas.

Con estas notaciones establecidas podemos pasar a las relaciones que el programa determina:

Tabla #.

Relaciones que el programa puede determinar y su significado.

Nombre del Conjunto	Significado
$A \cap B$	Líneas operativas en revisión.
$A - C$	Líneas activas sin fallos.
$B \cup C$	Líneas con incidencias.

Implementación y Pruebas.

En la implementación y pruebas se desarrolló el sistema TechFactory en C++, utilizando estructuras dinámicas y lógica proposicional. Mediante datos simulados se comprobó que el programa clasifica correctamente las líneas de producción y ejecuta adecuadamente las operaciones de conjuntos, la cola de revisión y el historial de operaciones.

Entorno de Desarrollo.

El sistema fue desarrollado en el lenguaje de programación C++, utilizando programación estructurada y estructuras dinámicas de datos.

Las principales librerías estándar utilizadas fueron:

- `<iostream>` → manejo de entrada y salida de datos.
- `<string>` → manipulación de cadenas de texto.
- `<sstream>` → procesamiento y conversión de datos.
- `<iomanip>` → formato de impresión en consola.
- `<cmath>` → operaciones matemáticas.

Casos de Prueba.

Para verificar el correcto funcionamiento del sistema se realizaron distintos escenarios simulados utilizando datos de entrada específicos.

Caso de prueba 1.

Dato	Valor
Producción actual	120
Meta diaria	100
Fallos técnicos	0
Supervisor válido	Sí

LÓGICA PROPOSICIONAL EN LÍNEAS DE PRODUCCIÓN

La línea se clasifica como “operativa”, debido a que cumple la meta, no presenta fallos y cuenta con validación del supervisor.

Caso de prueba 2.

Dato	Valor
Producción actual	80
Meta diaria	100
Fallos técnicos	0
Supervisor válido	Sí

La línea se clasifica como “bajo rendimiento”, porque no alcanza la meta diaria, aunque no presenta fallos.

Caso de prueba 3.

Dato	Valor
Producción actual	50
Meta diaria	100
Fallos técnicos	3
Supervisor válido	No

La línea se clasifica como detenida, ya que existen fallos mecánicos registrados.

Resultados Obtenidos.

Caso 1.

```

-----
Línea : Caso 1
-----
Producción actual : 120.0 uds
Meta diaria      : 100.0 uds
Fallos mecánicos : 0
Validación superv.: Sí

--- Proposiciones lógicas ---
P (cumple meta)   : VERDADERO
Q (sin fallos)    : VERDADERO
R (supervisor OK) : VERDADERO

--- Inferencia ---
Regla aplicada   :  $P \wedge Q \wedge R \rightarrow \text{OPERATIVA}$ 
Estado inferido  : [✓ OPERATIVA]
```

Evaluación lógica:

$P = \text{Verdadero} \rightarrow \text{cumple la meta.}$

$Q = \text{Verdadero} \rightarrow \text{no existen fallos.}$

$R = \text{Verdadero} \rightarrow \text{supervisor autorizado.}$

Como se cumple la regla:

$P \wedge Q \wedge R \rightarrow \text{OPERATIVA}$

Caso 2.

```
-----
Línea : Caso 2
-----
Producción actual : 80.0 uds
Meta diaria       : 100.0 uds
Fallos mecánicos  : 0
Validación superv.: Sí

--- Proposiciones lógicas ---
P (cumple meta)   : FALSO
Q (sin fallos)    : VERDADERO
R (supervisor OK) : VERDADERO

--- Inferencia ---
Regla aplicada    :  $\neg P \wedge Q \rightarrow \text{BAJO RENDIMIENTO}$ 
Estado inferido   : [ $\sim$  BAJO RENDIMIENTO]
```

Evaluación lógica:

- $P = \text{Falso} \rightarrow$ no cumple la meta.
- $Q = \text{Verdadero} \rightarrow$ no existen fallos.
- $R = \text{Verdadero} \rightarrow$ supervisor autorizado.

La línea no cumple la producción requerida, por lo que se aplica:

$\neg P \wedge Q \rightarrow \text{BAJO RENDIMIENTO}$

Caso 3.

```
-----
Línea : Caso 3
-----
Producción actual : 50.0 uds
Meta diaria       : 100.0 uds
Fallos mecánicos  : 3
Validación superv.: NO

--- Proposiciones lógicas ---
P (cumple meta)   : FALSO
Q (sin fallos)    : FALSO
R (supervisor OK) : FALSO

--- Inferencia ---
Regla aplicada    :  $\neg Q \rightarrow \text{DETENIDA}$ 
Estado inferido   : [ $\times$  DETENIDA]
```

Evaluación lógica:

- $P = \text{Falso} \rightarrow$ no cumple la meta.
- $Q = \text{Falso} \rightarrow$ existen fallos mecánicos.
- $R = \text{Falso} \rightarrow$ supervisor no válido.

Debido a que existen fallos mecánicos, el sistema aplica la regla:

$$\neg Q \rightarrow \text{DETENIDA}$$

Durante la ejecución del programa se obtuvieron resultados satisfactorios en cada una de las pruebas realizadas. El sistema logró:

- Registrar correctamente las líneas de producción.
- Evaluar las proposiciones lógicas P, Q y R.
- Inferir automáticamente el estado de cada línea.
- Gestionar adecuadamente la cola de revisión y el historial de operaciones.
- Mostrar un resumen general del estado de la planta.

Conclusiones.

- Al culminar este proyecto nos dimos cuenta de que la lógica proposicional es de mucha ayuda para la transformación de variables físicas (Control de Producción Industrial) en variables lógicas claras (P, Q, R), los cuales permitieron cumplir las tareas y las metas sin tener fallos mecánicos.
- El uso de las reglas lógicas también fue de mucha ayuda para la automatización de diagnósticos industriales ya que nos permitió clasificar el rendimiento de las líneas sin la intervención humana constante
- Por último, la teoría de conjuntos facilitó la gestión de datos facilitando así identificar las intersecciones críticas y uniones de incidencias, todo esto ayuda para tomar mejores decisiones

Referencias

Johnsonbaugh, R. (2005). Matemáticas discretas (6.^a ed.). Pearson Educación.